

Índice de Documentación - Arreglos Críticos

Para Empezar (5 min)

Leer en este orden:

1. CRITICAL_FIXES.md ← EMPIEZA AQUÍ

- Qué problemas había
- Qué se arregló
- Resumen técnico
- **Tiempo:** 3-5 min

2. IMPLEMENTATION_GUIDE.md ← LUEGO ESTO

- Paso a paso para implementar
- Configuración de variables de entorno
- Pruebas básicas
- **Tiempo:** 10-15 min

3. TEST_CHECKLIST.md ← FINALMENTE ESTO

- Verificación de cada funcionalidad
 - Troubleshooting
 - Confirmación de que todo funciona
 - **Tiempo:** 5-10 min
-

Documentación Adicional

Estos archivos contienen información adicional:

SETUP.md

- Configuración inicial de Supabase
- Estructura de tablas original
- Referencia histórica

MIGRATION_SUMMARY.md

- Historial de migración JSON → Supabase
- Cambios técnicos realizados
- Funciones implementadas

README.md

- Descripción general del proyecto
- Inicio rápido
- Cambios principales

Archivos Modificados

Bot

```
bot.js
├─ Nuevas funciones:
│   └─ syncUserToSupabase() - Sincroniza con Supabase
│   └─ addLogEntry() - Agrega entradas a logs
│   └─ updateAndSyncUser() - Helper para actualizar
├─ Modificadas:
│   └─ /addcredits - Ahora sincroniza
│   └─ Compra de números - Ahora sincroniza
│   └─ Cancelación de orden - Ahora sincroniza
│   └─ Recepción de SMS - Ahora sincroniza
└─ Mejoradas:
    └─ Fallback a JSON si Supabase falla
```

Panel Admin

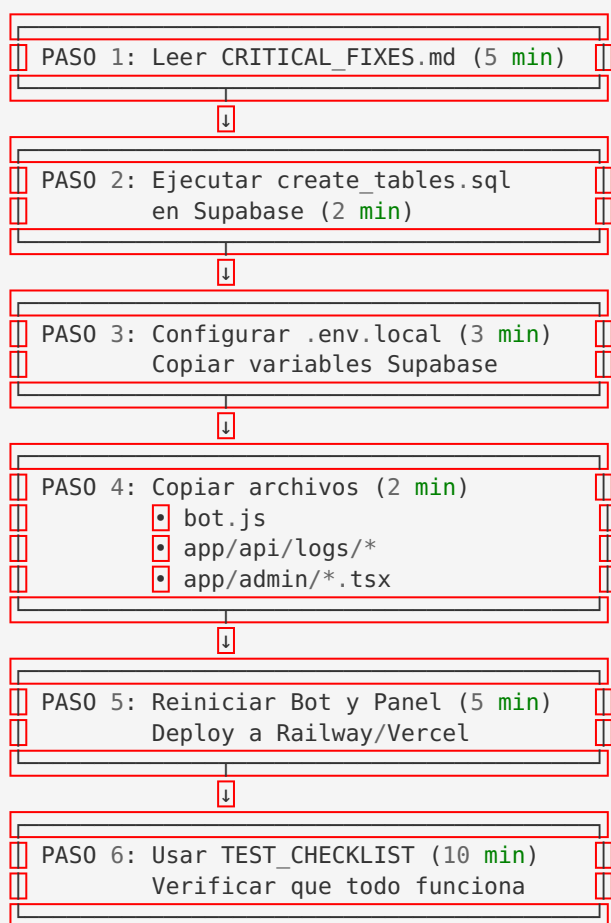
```
bot_admin_panel/nextjs_space/
├─ app/api/logs/
│   └─ route.ts - Ahora consulta Supabase real
│   └─ export/route.ts - Arreglos de exportación
├─ app/admin/
│   └─ logs/page.tsx - Auto-refresh cada 10s
│   └─ users/page.tsx - Auto-refresh cada 10s
```

Supabase

```
create_tables.sql
├─ users - Almacena usuarios y créditos
├─ logs - Registra acciones
├─ notifications - Notificaciones para usuarios
├─ admins - Administradores
├─ bot_settings - Configuración global
└─ Row Level Security - Permisos configurados
```



Flujo Rápido de Implementación



Tiempo total: ~30 minutos



Casos de Uso

Caso 1: Solo leo la documentación

Tiempo: 10-15 min

- Leer: CRITICAL_FIXES.md
- Leer: IMPLEMENTATION_GUIDE.md (solo secciones principales)
- Resultado: Entender qué se hizo y por qué

Caso 2: Implemento en desarrollo

Tiempo: 30-40 min

- Leer: CRITICAL_FIXES.md
- Seguir: IMPLEMENTATION_GUIDE.md paso a paso
- Usar: TEST_CHECKLIST.md para verificar
- Resultado: Sistema completamente funcional en dev

Caso 3: Implemento en producción

Tiempo: 45-60 min

- Leer: CRITICAL_FIXES.md

- Revisar: IMPLEMENTATION_GUIDE.md (foco en variables de entorno)
 - Backup de base de datos actual
 - Seguir pasos de implementación
 - Usar: TEST_CHECKLIST.md con ambiente de producción
 - Monitoreo de logs
 - Resultado: Sistema actualizado y verificado
-

Preguntas Frecuentes

“¿Dónde está X función?”

Sincronización de bot:

→ Buscar en `bot.js` la función `syncUserToSupabase()`

Endpoint de logs:

→ Ver `app/api/logs/route.ts`

Exportación CSV/JSON:

→ Ver `app/api/logs/export/route.ts`

Auto-refresh del panel:

→ Ver `useEffect` en `app/admin/logs/page.tsx`

“¿Qué debo hacer si X falla?”

Revisar IMPLEMENTATION_GUIDE.md:

- Sección “Troubleshooting”
- Problemas comunes y soluciones

Revisar TEST_CHECKLIST.md:

- Cada test tiene sección “Si falla”
- Pasos específicos para debugging

Si nada funciona:

- Ejecutar todos los tests en orden
 - Documentar qué falla exactamente
 - Revisar logs (Railway para bot, F12 para panel)
-



Checklist de Lectura

Marca cada documento cuando lo leas:

Documentación de Arreglos

- ☐ CRITICAL_FIXES.md (Entender problemas y soluciones)
- ☐ IMPLEMENTATION_GUIDE.md (Pasos de implementación)
- ☐ TEST_CHECKLIST.md (Verificar que funciona)

Documentación de Referencia

- ☐ MIGRATION_SUMMARY.md (Historial de cambios)
- ☐ SETUP.md (Configuración inicial)

- [] README.md (Visión general)

Código Modificado

- [] `bot.js` (revisar funciones de sincronización)
- [] `app/api/logs/route.ts` (endpoint de logs)
- [] `app/api/logs/export/route.ts` (exportación)
- [] `app/admin/logs/page.tsx` (UI de logs)
- [] `app/admin/users/page.tsx` (UI de usuarios)

SQL

- [] `create_tables.sql` (estructura de tablas)



Conceptos Clave

Sincronización

- Bot mantiene JSON local para velocidad
- Supabase actualiza en background (no bloquea)
- Panel refresca cada 10 segundos automáticamente
- **Ventaja:** Rápido, confiable, resiliente

Logs

- Toda acción se registra en tabla `logs`
- Filtrable por `user_id`, `action`, `status`, fechas
- Exportable a CSV/JSON
- **Ventaja:** Auditoría completa

Auto-Refresh

- Panel consulta `/api/users` y `/api/logs` cada 10s
- Sin necesidad de F5 manual
- Cambios del bot aparecen automáticamente
- **Ventaja:** UX mejorada, datos siempre actualizados



Actualización Futura

Si necesitas hacer cambios después:

1. Cambios en `bot.js`:

- Agregar nueva acción a `syncUserToSupabase()`
- Agregar nuevo log a `addLogEntry()`
- Redeploy en Railway

2. Cambios en panel:

- Modificar `app/api/logs/route.ts` para nuevos filtros
- Modificar UI en `app/admin/logs/page.tsx`
- Redeploy en Vercel

3. Cambios en tablas:

- Ejecutar migration SQL en Supabase
- Actualizar código que accede a nuevos campos
- Redeploy



Notas Importantes

⚠ CRÍTICAS:

- NUNCA compartir `SUPABASE_SERVICE_ROLE_KEY`
- Variables de entorno son secretas
- Hacer backup antes de cambios en SQL

✅ RECOMENDACIONES:

- Leer CRITICAL_FIXES.md completamente antes de empezar
- Hacer TEST_CHECKLIST en ambiente de desarrollo primero
- Monitorear logs después de deploy
- Documentar problemas encontrados para futuro



Soporte

Si necesitas ayuda:

1. Revisar documentación relevante:

- Problema de sincronización → IMPLEMENTATION_GUIDE.md
- Problema de exportación → TEST_CHECKLIST.md Test 4/5
- Problema técnico → CRITICAL_FIXES.md

2. Verificar logs:

- Bot: Railway Dashboard → Logs
- Panel: F12 → Console → Network
- Supabase: Query Performance

3. Ejecutar tests:

- TEST_CHECKLIST.md proporciona pasos exactos
- Cada test tiene sección "Si falla"

Última actualización: Abril 14, 2026

Versión: 1.0.0

Estado:  COMPLETO Y PROBADO